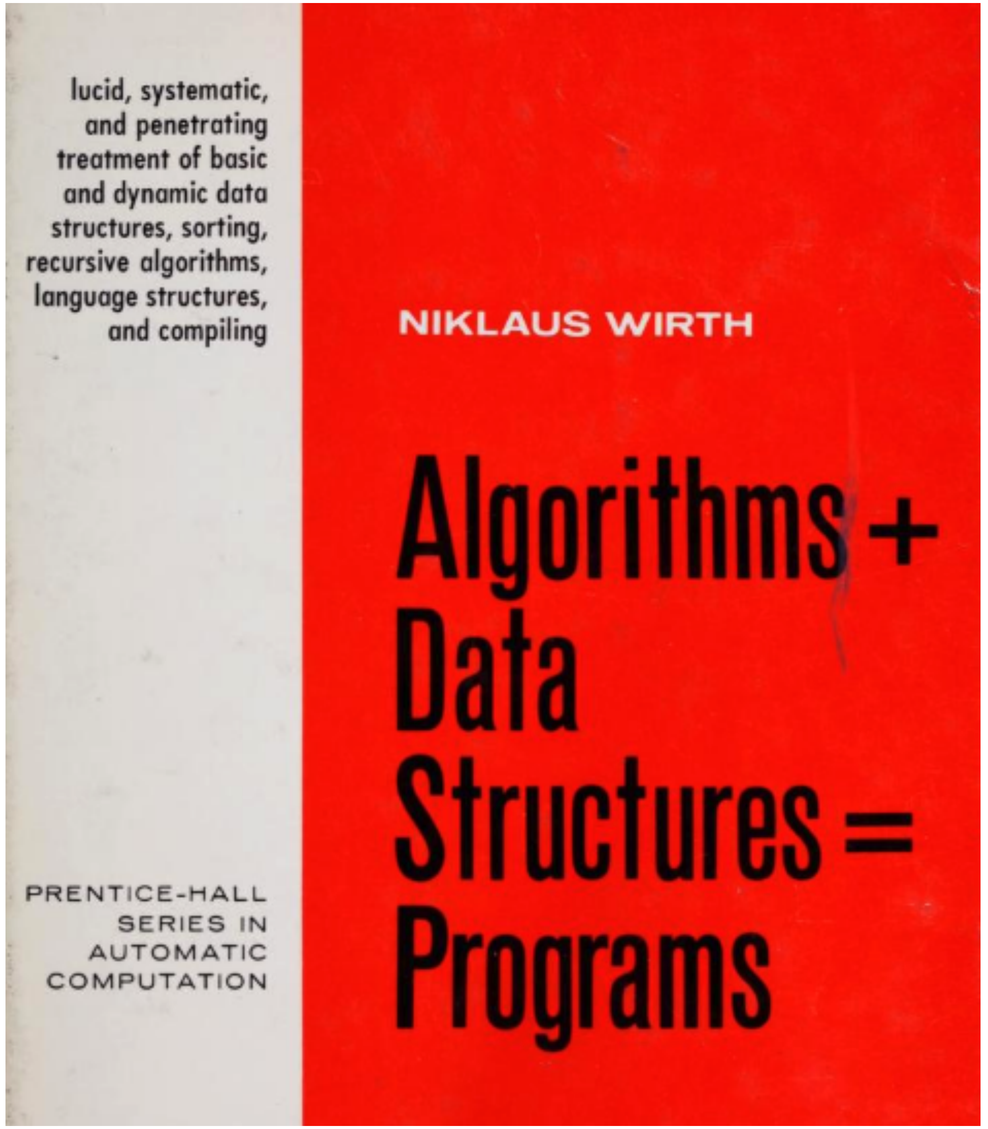
Structural design with Alloy


lucid, systematic, and penetrating treatment of basic and dynamic data structures, sorting, recursive algorithms, language structures, and compiling
NIKLAUS WIRTH
Algorithms + Data Structures = Programs
PRENTICE-HALL SERIES IN AUTOMATIC COMPUTATION

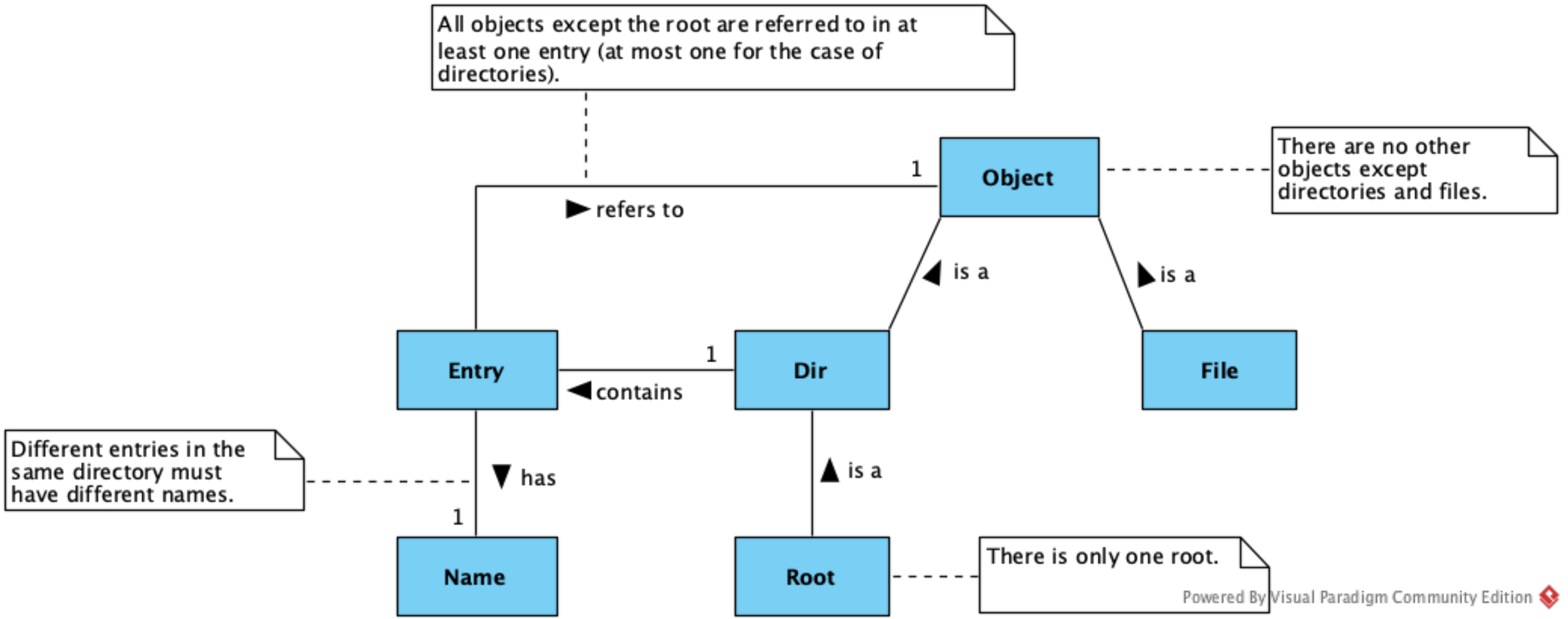
Software structures

- Data structures
- Database schemas
- Architectures
- Network topologies
- Ontologies
- Domain models

Structural design

- Understand entities and their relationships
- Elicit requirements
- Explore alternatives

Domain modeling *à la* UML



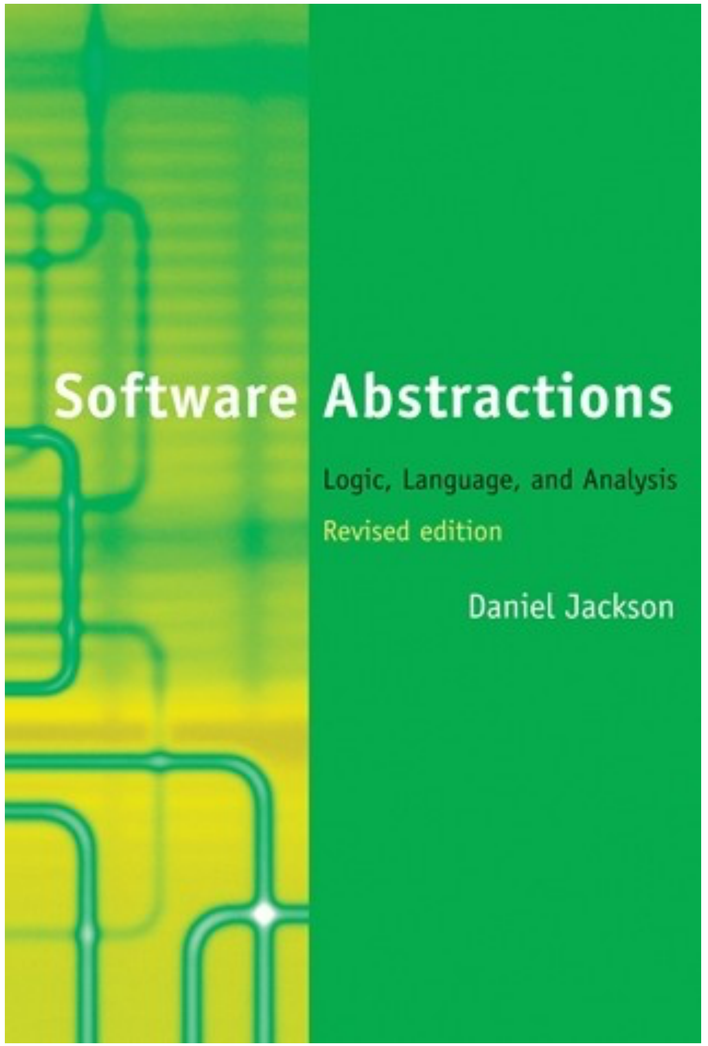
Requirement elicitation

- Are the requirements consistent?
- Any forgotten or redundant requirements?
- Do the requirements entail all the expected properties?

Alloy

Software design with Alloy

- Alloy is a formal modeling language
- Based on relational logic, an extension of first-order logic
- Models can be automatically analyzed
- Tailored for abstraction - everything is a relation!

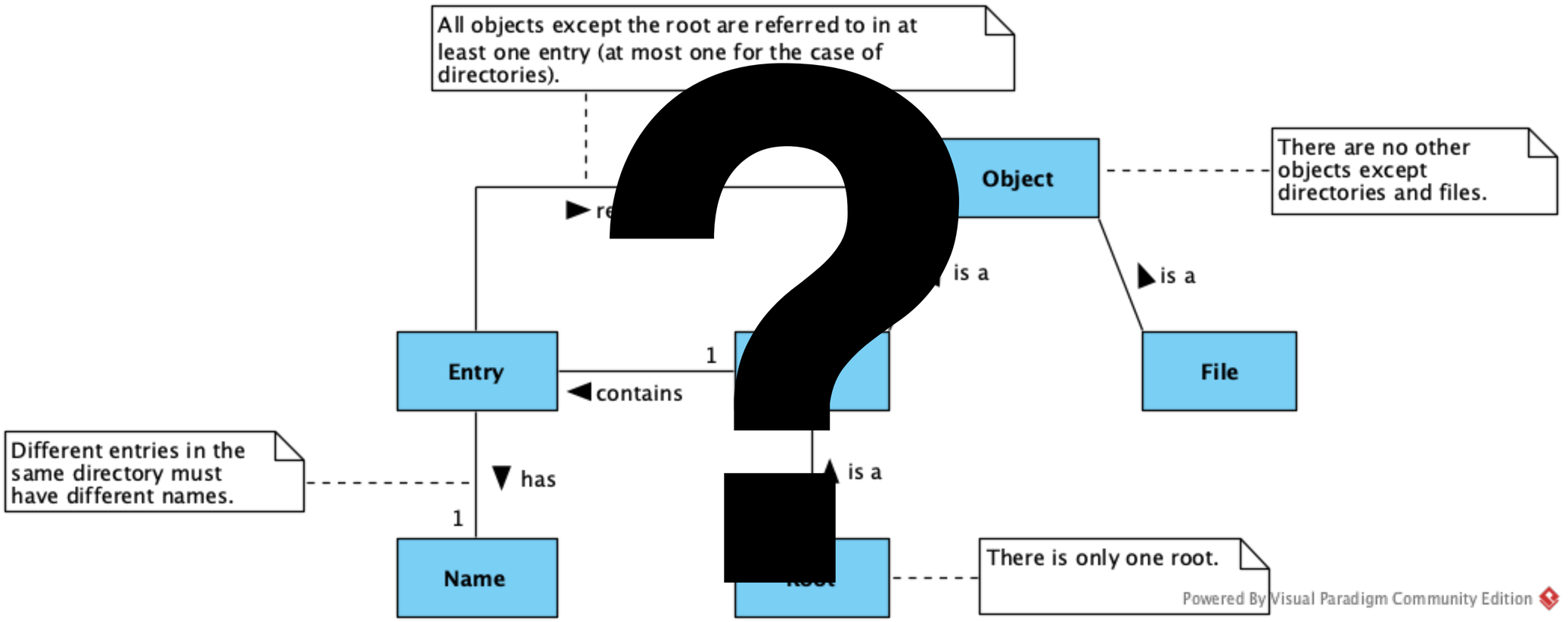

Software Abstractions
Logic, Language, and Analysis
Revised edition
Daniel Jackson

“The core of software development [...] is the *design* of abstractions. An abstraction is [...] an idea reduced to its essential form.”



–Daniel Jackson

Domain modeling with Alloy



First-order logic

Signatures

- Unary predicates are known as *signatures* and are declared with **`sig`**
- Signatures are inhabited by *atoms* from a finite *domain* of discourse
- Signatures can be top-level, extensions, or subsets
- Top-level and extension signatures are disjoint
- Signatures can be abstract, only containing atoms in the extensions
- Signatures can have a multiplicity (**`lone`**, **`some`**, **`one`**)

Top-level signatures

```
sig Object {}
sig Entry {}
sig Name {}
```

$\text{Object} \subseteq D$

$\text{Entry} \subseteq D$

$\text{Name} \subseteq D$

$\forall x \,.\, \neg(\text{Object}(x) \wedge \text{Entry}(x))$

$\forall x \,.\, \neg(\text{Object}(x) \wedge \text{Name}(x))$

$\forall x \,.\, \neg(\text{Name}(x) \wedge \text{Entry}(x))$

Extension signatures

```
sig Dir extends Object {}
sig File extends Object {}
```

$\text{Dir} \subseteq D$

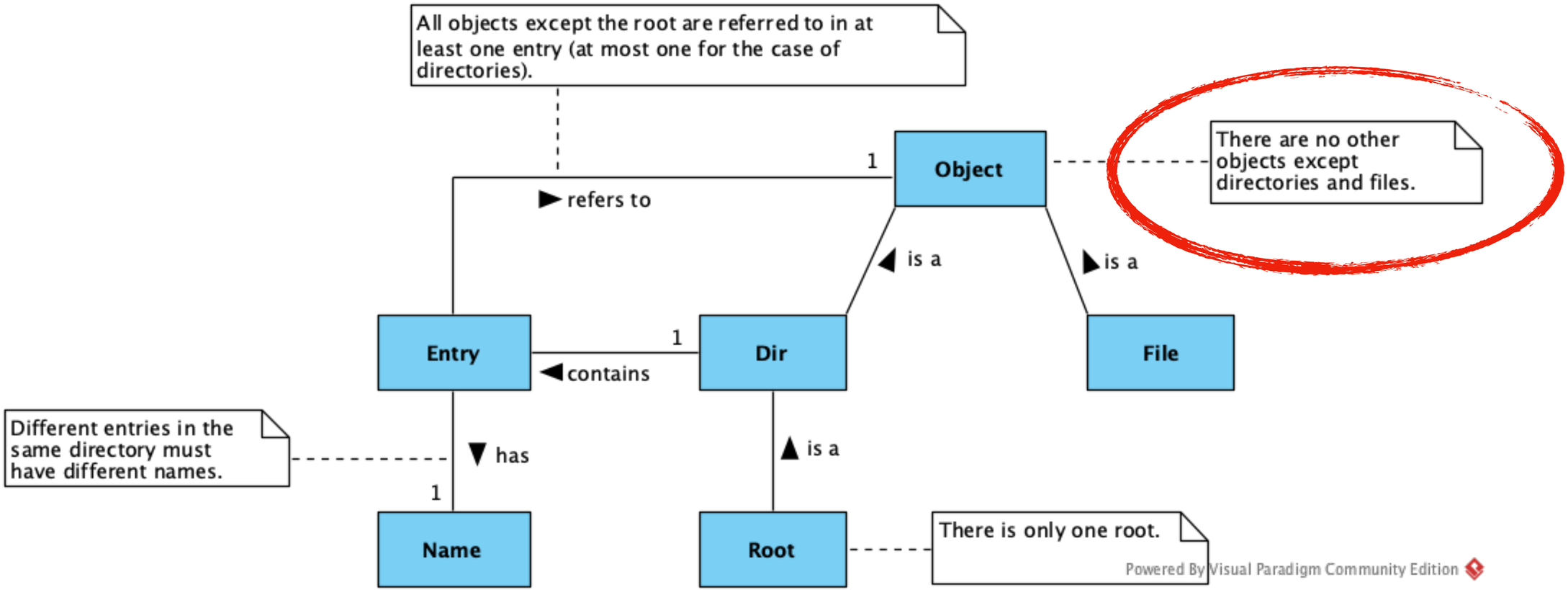
$\text{File} \subseteq D$

$\forall x \,.\, \text{File}(x) \rightarrow \text{Object}(x)$

$\forall x \,.\, \text{Dir}(x) \rightarrow \text{Object}(x)$

$\forall x \,.\, \neg(\text{File}(x) \wedge \text{Dir}(x))$

Object is abstract



Abstract signatures

```
abstract sig Object {}
sig Dir extends Object {}
sig File extends Object {}
```

$$\forall x : \text{Object} \,.\, \text{Dir}(x) \vee \text{File}(x)$$

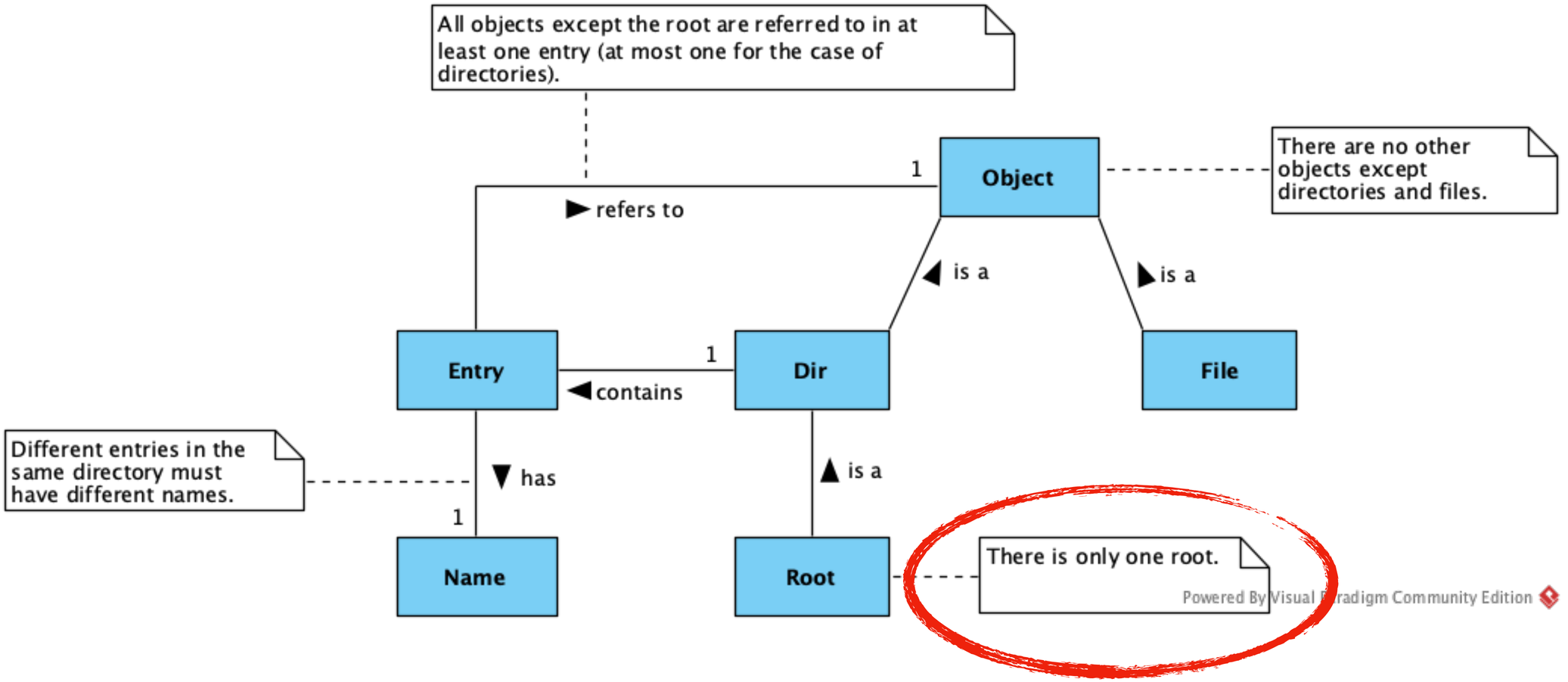
Subset signatures

```
sig Root in Dir {}
```

$$\text{Root} \subseteq D$$

$$\forall x \,.\, \text{Root}(x) \rightarrow \text{Dir}(x)$$

There is only one root



Signature multiplicities

```
one sig Root in Dir {}
```

$\exists x \,.\, \text{Root}(x)$

$\forall x, y : \text{Root} \,.\, x = y$

Fields

- Predicates of arity 2 or more are known as *fields*
- Fields are inhabited by *tuples* of atoms
- Must be declared inside the *domain* signature
- Multiplicities (**`set`**, **`lone`**, **`some`**, **`one`**) can be imposed on the targets
- If no multiplicity is imposed the default is **`one`**

Fields

```
sig Dir {
  contains : set Entry
}
```

contains $\subseteq D \times D$

$\forall x, y \,.\, \text{contains}(x, y) \rightarrow \text{Dir}(x) \wedge \text{Entry}(y)$

Fields

```
sig Entry {
  refersTo : one Object,
  has : one Name
}
```

$$\text{refersTo} \subseteq D \times D$$

$$\forall x, y \,.\, \text{refersTo}(x, y) \rightarrow \text{Entry}(x) \wedge \text{Object}(y)$$

$$\forall x : \text{Entry} \,.\, \exists y \,.\, \text{refersTo}(x, y)$$

$$\forall x, y, z \,.\, \text{refersTo}(x, y) \wedge \text{refersTo}(x, z) \rightarrow y = z$$

$$\text{has} \subseteq D \times D$$

$$\forall x, y \,.\, \text{has}(x, y) \rightarrow \text{Entry}(x) \wedge \text{Name}(y)$$

$$\forall x : \text{Entry} \,.\, \exists y \,.\, \text{has}(x, y)$$

$$\forall x, y, z \,.\, \text{has}(x, y) \wedge \text{has}(x, z) \rightarrow y = z$$

Facts

- *Facts* specify assumptions

```
fact { φ }
```

- Facts can be named

```
fact Name { φ }
```

- A single fact can have several constraints, one per line

```
fact {
  φ
  ψ
}
```

FOL vs Alloy

$\neg\phi$	`!` ϕ
$\phi \wedge \psi$	ϕ `&&` ψ
$\phi \vee \psi$	ϕ `||` ψ
$\phi \rightarrow \psi$	ϕ `=>` ψ
$(\phi \wedge \psi) \vee (\neg\phi \wedge \theta)$	ϕ `=>` ψ **`else`** θ
$\phi \leftrightarrow \psi$	ϕ `<=>` ψ

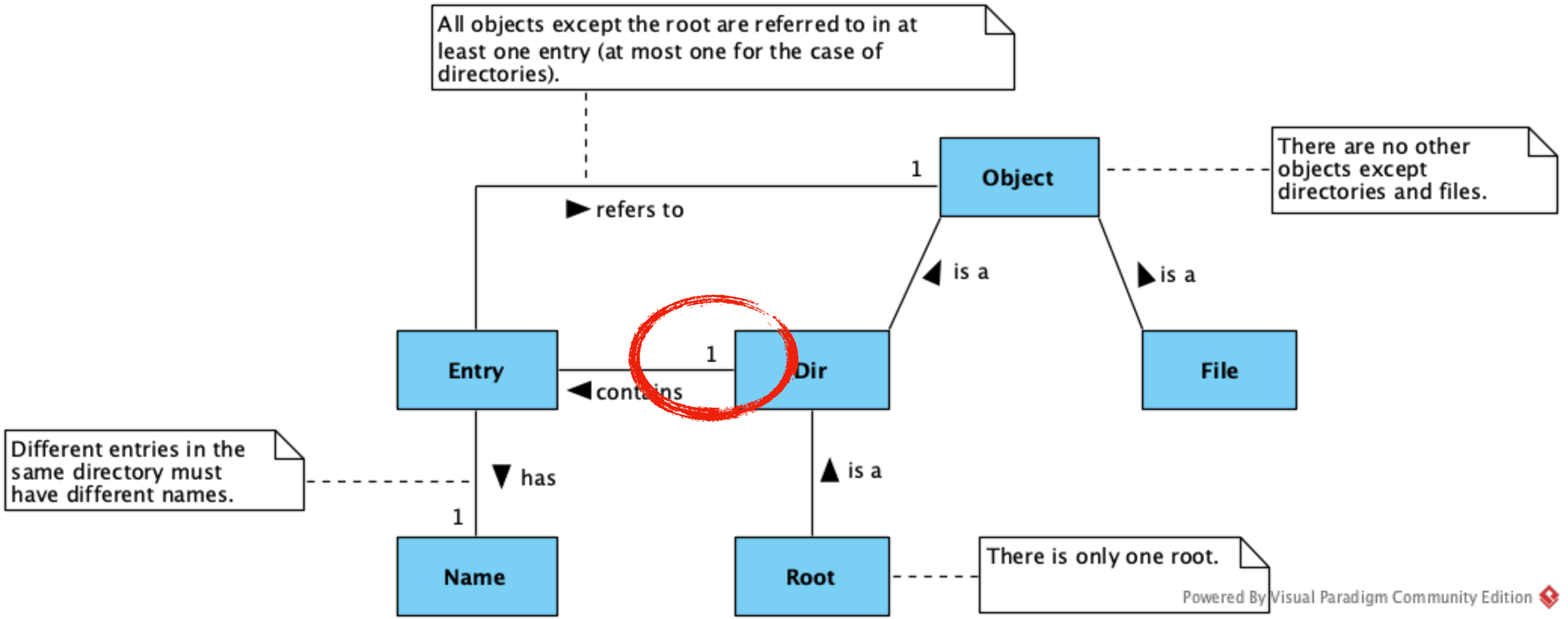
FOL vs Alloy

$\neg\phi$	**not** ϕ
$\phi \wedge \psi$	ϕ **and** ψ
$\phi \vee \psi$	ϕ **or** ψ
$\phi \rightarrow \psi$	ϕ **implies** ψ
$(\phi \wedge \psi) \vee (\neg\phi \wedge \theta)$	ϕ **implies** ψ **else** θ
$\phi \leftrightarrow \psi$	ϕ **iff** ψ

FOL vs Alloy

$x = y$	x `=` y
$A(x)$	x `in` A
$R(x, y)$	x`->`y `in` R
$\forall x \,.\, A(x) \rightarrow \phi$	`all` x `:` A `|` ϕ
$\exists x \,.\, A(x) \wedge \phi$	`some` x `:` A `|` ϕ

Each entry is contained in one directory



Each entry is contained in one directory

```
fact {
   // Each entry is contained in one directory
   all x : Entry | some y : Dir | y->x in contains
   all x : Entry, y,z : Dir {
     y->x in contains and z->x in contains implies y = z
   }
}
```

Commands

- Alloy has two types of analysis *commands*:
  - **`run`** `{ φ }` asks for an *example* that satisfies all facts and φ
  - **`check`** `{ φ }` asks for a *counter-example* that satisfies all facts but refutes assertion φ
- Likewise facts, commands can be named and can have several constraints, one per line
- In the visualizer it is possible to ask for more examples or counter-examples by pressing *New*

Instances

- Both examples and counter-examples are first-order structures
- In Alloy first-order structures are known as *instances*
- An instance is a valuation to all the signatures and fields
- In an instance “everything is a relation”
  - Signatures are unary relations (sets of unary tuples)
  - Constants are singleton unary relations (sets with one unary tuple)
- By default instances are depicted as graphs

Scopes

- To ensure decidability commands have a *scope*
- The scope imposes a limit on the size of the (finite) domain the Analyzer will exhaustively explore
- The default scope imposes a limit of 3 atoms per top-level signature
- **`for`** can be used to specify a different scope for top-level signatures
- **`but`** can be used to specify different scopes for specific signatures
- **`exactly`** can be used to specify exact scopes

The small scope hypothesis

- If **`run { φ }`** returns an instance then φ is *consistent*, else φ **may be *inconsistent***

  - Could be consistent with a bigger scope!

- If **`check { φ }`** returns an instance then φ is *invalid,* else φ **may be *valid***

  - Could be invalid with a bigger scope!!!

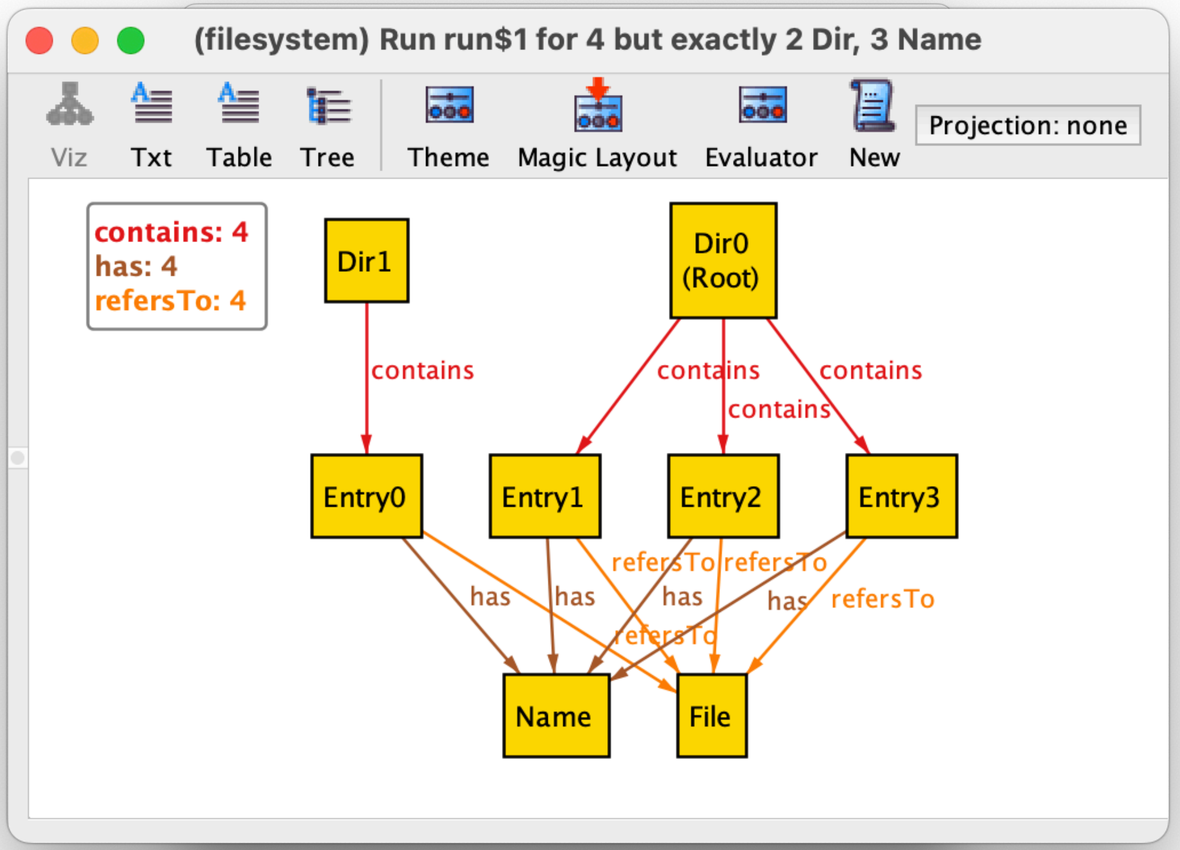
- Anecdotical evidence suggests that most invalid assertions (or consistent predicates) can be refuted (or witnessed) with a small scope

Atoms

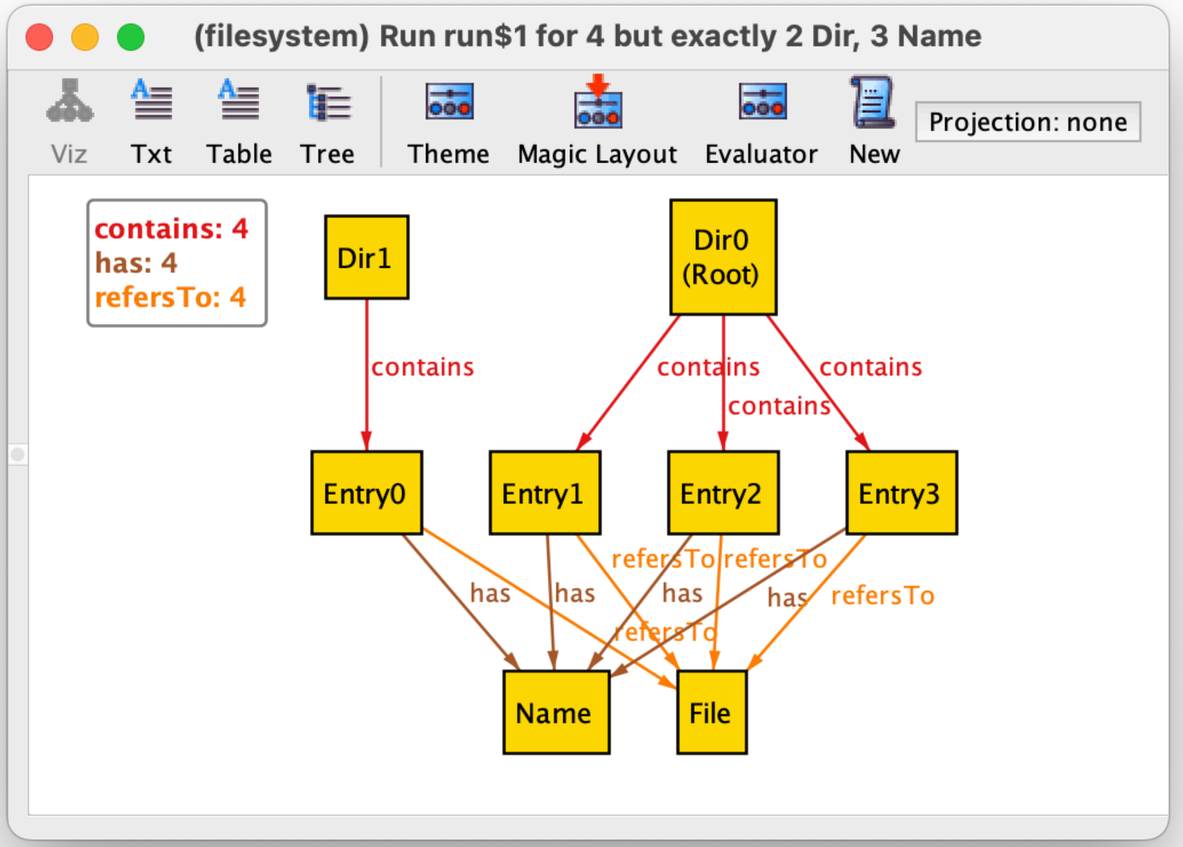
- The universe of discourse contains *atoms*
- Atoms are *uninterpreted* (no semantics)
- Named automatically according to the respective signatures
- Two instances are *isomorphic* (or *symmetric*) if they are equal modulo renaming
- The analysis implements a *symmetry breaking* mechanism to avoid returning isomorphic instances

A simple command

```
run {} for 4 but exactly 2 Dir, 3 Name
```



Instances as graphs



Instances as relations

```
Object   = {(Dir0),(Dir1),(File)}
Dir      = {(Dir0),(Dir1)}
File     = {(File)}
Root     = {(Dir0)}
Entry    = {(Entry0),(Entry1),(Entry2),(Entry3)}
Name     = {(Name)}
contains = {(Dir1,Entry0),(Dir0,Entry1),(Dir0,Entry2),(Dir0,Entry3)}
refersTo = {(Entry0,File),(Entry1,File),(Entry2,File),(Entry3,File)}
has      = {(Entry0,Name),(Entry1,Name),(Entry2,Name),(Entry3,Name)}
```

Instances as tables

Object
Dir0
Dir1
File

Dir
Dir0
Dir1

Root
Dir0

File
File

Name
Name

Entry
Entry0
Entry1
Entry2
Entry3

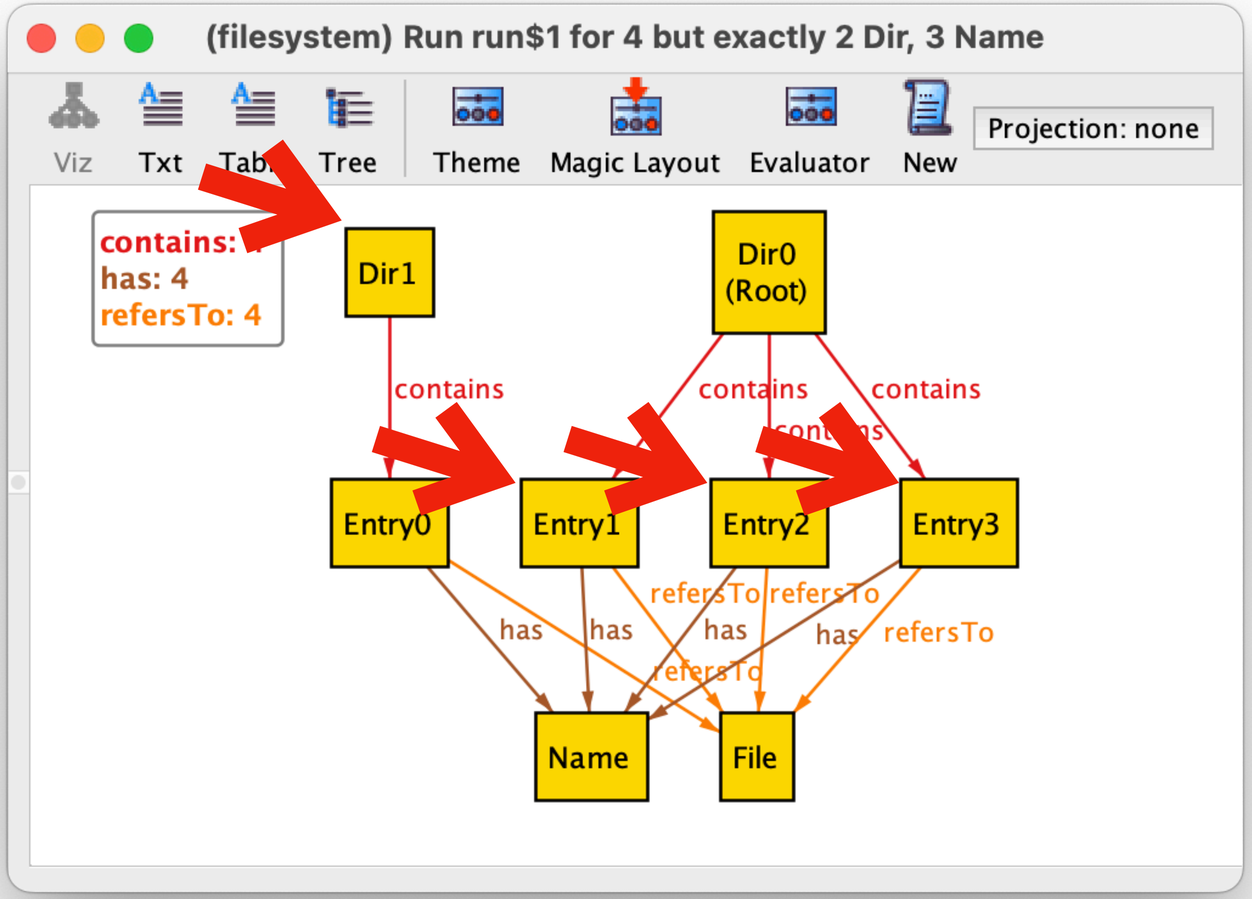
contains	
Dir1	Entry0
Dir0	Entry1
Dir0	Entry2
Dir0	Entry3

refersTo	
Entry0	File
Entry1	File
Entry2	File
Entry3	File

has	
Entry0	Name
Entry1	Name
Entry2	Name
Entry3	Name

Additional requirements

- All objects except the root are referred to in at least one entry (at most one for the case of directories)
- Different entries in a directory must have different names



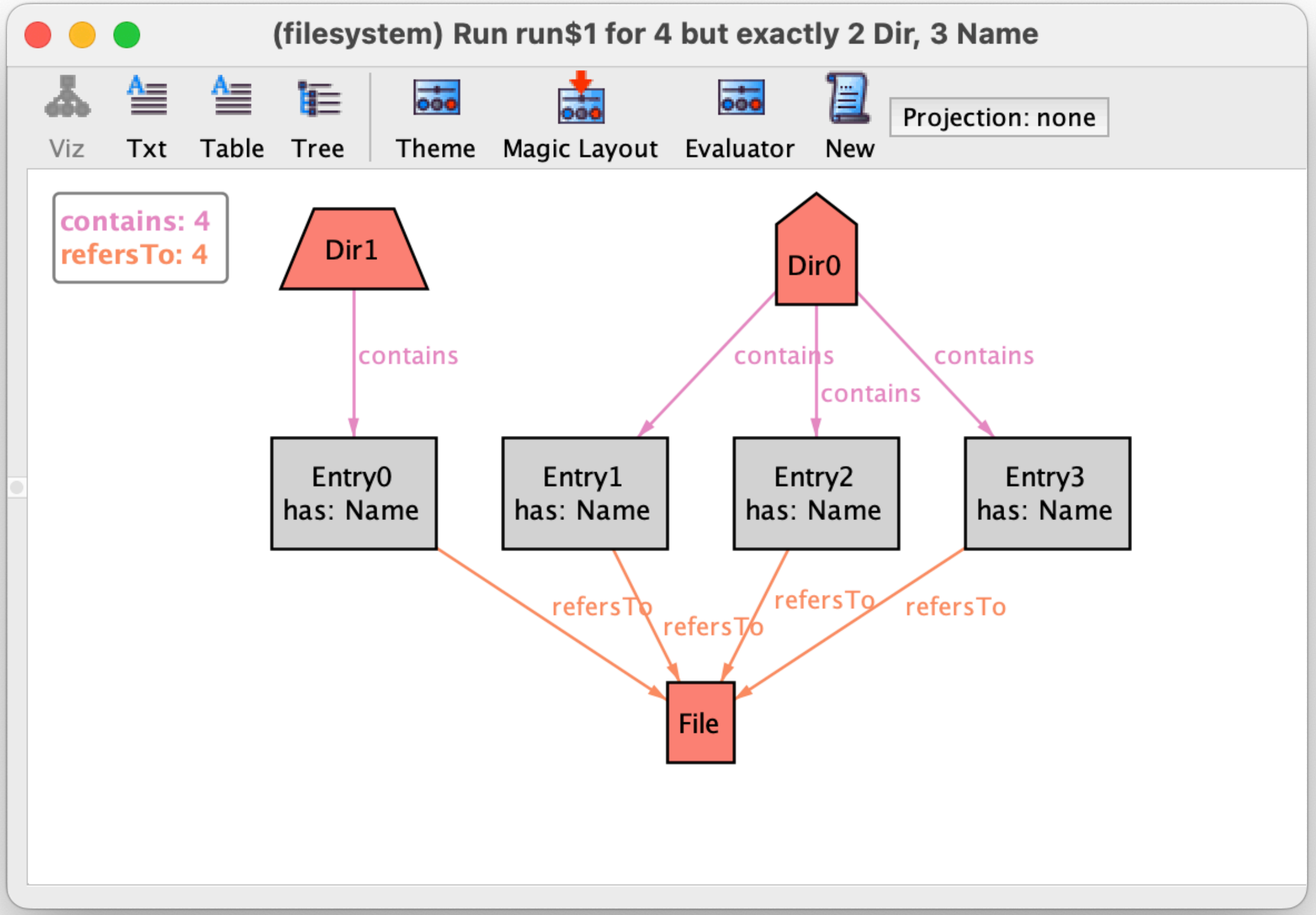
Themes

- The visualizer theme can be customized
- Customization can ease the understanding and help validate the model
- It is possible to customize colors, shapes, visibility, …

Theme customization



Theme customization



Requirements breakdown

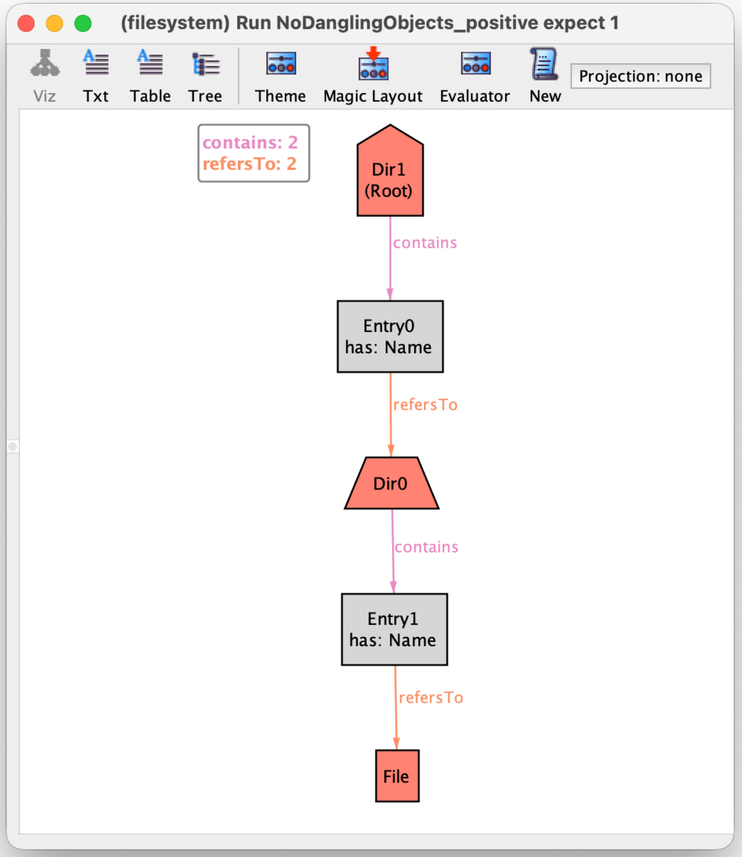
- All objects except the root are referred to in at least one entry (at most one for the case of directories)
  - All directories are referred to in at most one entry
  - The root is not referred in any entry
  - All non-root objects are referred to in at least one entry
- Different entries in a directory must have different names

Scenario driven specification

- Before specifying each requirement write some scenarios for validation
- Scenarios can be specified with **`run`** commands
- Include at least one positive and one negative scenario
  - Use **`expect`** keyword to document expectation
  - Verify that positive scenarios are SAT
  - Verify that negative scenarios are UNSAT
- Recheck all scenarios every time facts are added or changed

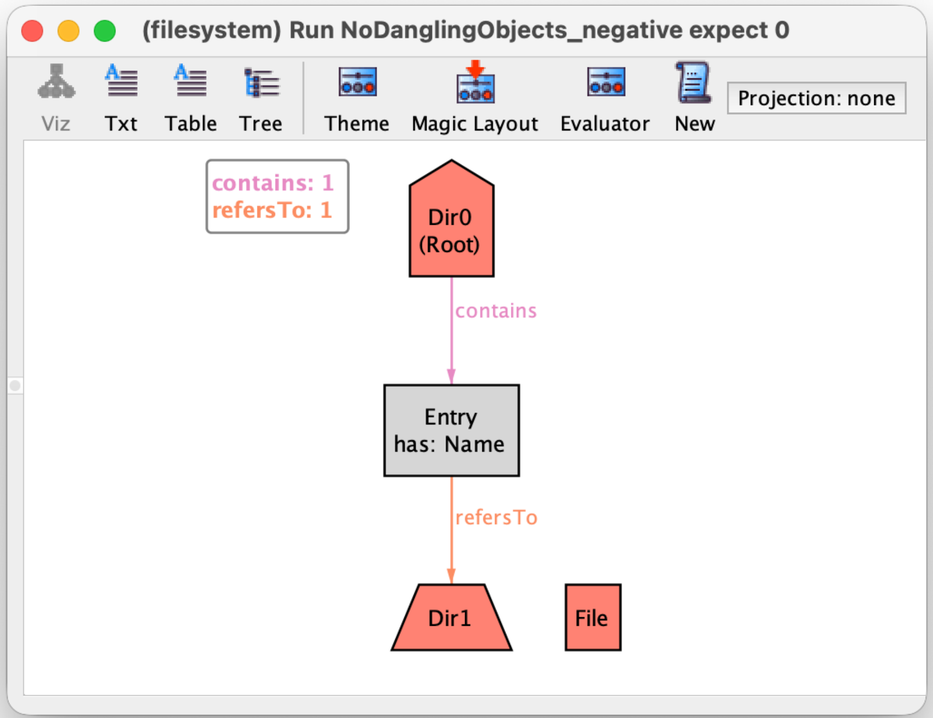
All non-root objects are referred to in at least one entry

```
run NoDanglingObjects_positive {
  some r : Root, d : Dir, f : File, disj e1,e2 : Entry, n : Name {
      Dir      = r + d
      Root     = r
      File     = f
      Entry    = e1 + e2
      Name     = n
      contains = r->e1 + d->e2
      refersTo = e1->d + e2->f
      has      = e1->n + e2->n
  }
} expect 1
```



All non-root objects are referred to in at least one entry

```
run NoDanglingObjects_negative {
  some r : Root, d : Dir, f : File, e : Entry, n : Name {
    Dir      = r + d
    Root     = r
    File     = f
    Entry    = e
    Name     = n
    contains = r->e
    refersTo = e->d
    has      = e->n
  }
} expect 0
```



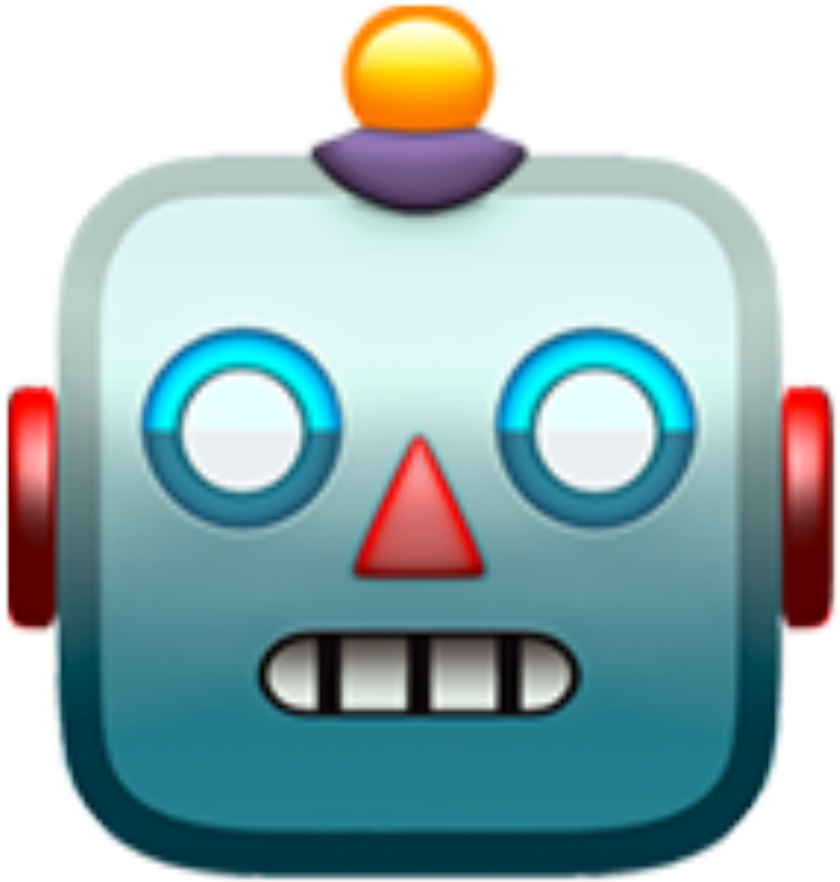
All non-root objects are referred to in at least one entry

```
fact NoDanglingObjects {
   all x : Object | x not in Root implies some y : Entry | y->x in refersTo
}
```

The remaining requirements

```
// All directories are referred to in at most one entry
fact NoSharedDirs {
    all x : Dir, y,z : Entry | y->x in refersTo and z->x in refersTo implies y = z
}
// The root is not referred in any entry
fact RootIsDangling {
    all x : Entry, y : Root | x->y not in refersTo
}
// Different entries in a directory must have different names
fact UniqueNames {
    all x : Dir, y,z : Entry, w : Name {
        x->y in contains and x->z in contains and y->w in has and z->w in has implies y = z
    }
}
```





Specification Engineering with LLMs

- Scenario generation
  - Safe, because user can easily review the scenarios
- Autoformalization
  - Difficult to review the result and dangerous if trusted blindly
  - Risk can be reduced if paired with scenarios, equivalence checking between alternatives (ideally proposed by different models), …
  - But there will always be some risk, we need to understand the specs!

Relational logic

Relational logic

- *Relational logic* extends FOL
- Adds operators to combine predicates (relations) into *terms*
- Terms denote derived relations
- Adds transitive closure, which cannot be expressed in FOL

Syntax

$$x, y, z, \ldots \in \mathscr{X}$$
$$P, Q, R, \ldots \in \mathscr{P}$$
$$t, u, \ldots \in \mathbf{Term}_{\mathscr{P}}$$
$$\phi, \psi, \ldots \in \mathbf{Form}_{\mathscr{P}}$$

$$\begin{aligned}
\phi, \psi \doteq\ & t \subseteq u \\
\mid\ & \top \\
\mid\ & \bot \\
\mid\ & (\neg\phi) \\
\mid\ & (\phi \wedge \psi) \\
\mid\ & (\phi \vee \psi) \\
\mid\ & (\phi \to \psi) \\
\mid\ & (\phi \leftrightarrow \psi) \\
\mid\ & (\forall x\,.\,\phi) \\
\mid\ & (\exists x\,.\,\phi)
\end{aligned}$$

$$\begin{aligned}
t, u \doteq\ & x, y, z, \ldots \\
\mid\ & P, Q, R, \ldots \\
\mid\ & \varnothing \\
\mid\ & \mathbf{U} \\
\mid\ & \mathbf{id} \\
\mid\ & t \cup u \\
\mid\ & t \cap u \\
\mid\ & t \backslash u \\
\mid\ & t \times u \\
\mid\ & t \bullet u \\
\mid\ & t^{\circ} \\
\mid\ & t^{+}
\end{aligned}$$

Formula semantics

$\mathscr{M}, \mathscr{A} \vDash \top$

$\mathscr{M}, \mathscr{A} \nvDash \bot$

$\mathscr{M}, \mathscr{A} \vDash t \subseteq u$	iff	$\mathscr{V}(t)$ is a subset or equal to $\mathscr{V}(u)$
$\mathscr{M}, \mathscr{A} \vDash \neg\phi$	iff	$\mathscr{M}, \mathscr{A} \nvDash \phi$
$\mathscr{M}, \mathscr{A} \vDash \phi \wedge \psi$	iff	$\mathscr{M}, \mathscr{A} \vDash \phi$ and $\mathscr{M}, \mathscr{A} \vDash \psi$
$\mathscr{M}, \mathscr{A} \vDash \phi \vee \psi$	iff	$\mathscr{M}, \mathscr{A} \vDash \phi$ or $\mathscr{M}, \mathscr{A} \vDash \psi$
$\mathscr{M}, \mathscr{A} \vDash \phi \rightarrow \psi$	iff	$\mathscr{M}, \mathscr{A} \nvDash \phi$ or $\mathscr{M}, \mathscr{A} \vDash \psi$
$\mathscr{M}, \mathscr{A} \vDash \phi \leftrightarrow \psi$	iff	$\mathscr{M}, \mathscr{A} \vDash \phi$ iff $\mathscr{M}, \mathscr{A} \vDash \psi$
$\mathscr{M}, \mathscr{A} \vDash \forall x\,.\,\phi$	iff	$\mathscr{M}, \mathscr{A}[x \mapsto a] \vDash \phi$ for all $a \in D$
$\mathscr{M}, \mathscr{A} \vDash \exists x\,.\,\phi$	iff	$\mathscr{M}, \mathscr{A}[x \mapsto a] \vDash \phi$ for some $a \in D$

Term semantics

$$\mathscr{V}(x) \doteq \{(\mathscr{A}(x))\}$$
$$\mathscr{V}(R) \doteq I(R)$$
$$\mathscr{V}(\varnothing) \doteq \{\,\}$$
$$\mathscr{V}(\mathbf{U}) \doteq \{(x) \mid x \in D\}$$
$$\mathscr{V}(\mathbf{id}) \doteq \{(x, x) \mid x \in D\}$$
$$\mathscr{V}(t \cup u) \doteq \{(x_1, \ldots, x_{|t|}) \mid (x_1, \ldots, x_{|t|}) \in \mathscr{V}(t) \vee (x_1, \ldots, x_{|t|}) \in \mathscr{V}(u)\}$$
$$\mathscr{V}(t \cap u) \doteq \{(x_1, \ldots, x_{|t|}) \mid (x_1, \ldots, x_{|t|}) \in \mathscr{V}(t) \wedge (x_1, \ldots, x_{|t|}) \in \mathscr{V}(u)\}$$
$$\mathscr{V}(t \backslash u) \doteq \{(x_1, \ldots, x_{|t|}) \mid (x_1, \ldots, x_{|t|}) \in \mathscr{V}(t) \wedge (x_1, \ldots, x_{|t|}) \notin \mathscr{V}(u)\}$$
$$\mathscr{V}(t \times u) \doteq \{(x_1, \ldots, x_{|t|}, y_1, \ldots, y_{|u|}) \mid (x_1, \ldots, x_{|t|}) \in \mathscr{V}(t) \wedge (y_1, \ldots, y_{|u|}) \in \mathscr{V}(u)\}$$
$$\mathscr{V}(t \bullet u) \doteq \{(x_1, \ldots, x_{|t|-1}, y_2, \ldots, y_{|u|}) \mid \exists x_{|t|} \,.\, \exists y_1 \,.\, (x_1, \ldots, x_{|t|}) \in \mathscr{V}(t) \wedge (y_1, \ldots, y_{|u|}) \in \mathscr{V}(u) \wedge x_{|t|} = y_1\}$$
$$\mathscr{V}(t^\circ) \doteq \{(x_1, \ldots, x_{|t|}) \mid (x_{|t|}, \ldots, x_1) \in \mathscr{V}(t)\}$$
$$\mathscr{V}(t^+) \doteq \mathscr{V}(t \cup t \bullet t \cup t \bullet t \bullet t \cup \ldots)$$

FOL $\subseteq$ RL

$x = y$

$P(x, y)$

$x \subseteq y$

$x \times y \subseteq P$

$\forall x \,.\, \forall y \,.\, \mathsf{bff}(x, y) \rightarrow \mathsf{friend}(x, y)$

$\forall x \,.\, \neg \mathsf{friend}(x, x)$

$\forall x \,.\, \forall y \,.\, x \neq y \rightarrow \mathsf{friend}(x, y)$

$\forall x \,.\, \forall y \,.\, x \times y \subseteq \mathsf{bff} \rightarrow x \times y \subseteq \mathsf{friend}$

$\forall x \,.\, \neg(x \times x \subseteq \mathsf{friend})$

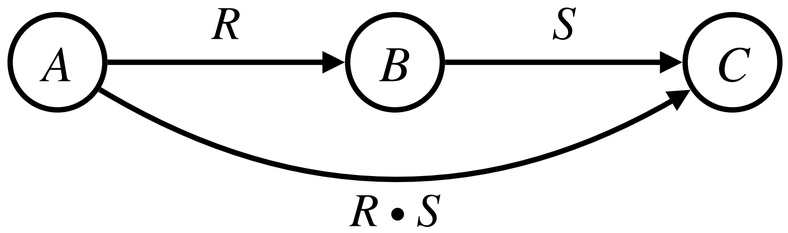
$\forall x \,.\, \forall y \,.\, \neg(x \subseteq y) \rightarrow x \times y \subseteq \mathsf{friend}$

FOL vs RL

$\forall x \,.\, \forall y \,.\, \mathsf{bff}(x, y) \rightarrow \mathsf{friend}(x, y)$	$\mathsf{bff} \subseteq \mathsf{friend}$
$\forall x \,.\, \forall y \,.\, \mathsf{friend}(x, y) \rightarrow \mathsf{friend}(y, x)$	$\mathsf{friend} \subseteq \mathsf{friend}^\circ$
$\forall x \,.\, \neg \mathsf{friend}(x, x)$	$\mathsf{friend} \cap \mathbf{id} \subseteq \varnothing$
$\forall x \,.\, \forall y \,.\, \mathsf{Ann}(x) \wedge \mathsf{Student}(y) \rightarrow \mathsf{friend}(x, y)$	$\mathsf{Ann} \times \mathsf{Student} \subseteq \mathsf{friend}$
$\forall x \,.\, \forall y \,.\, x \neq y \rightarrow \mathsf{friend}(x, y)$	$(\mathbf{U} \times \mathbf{U}) \backslash \mathbf{id} \subseteq \mathsf{friend}$

Composition

$$R \subseteq A \times B \ \wedge \ S \subseteq B \times C \ \rightarrow \ R \bullet S \subseteq A \times C$$



$$\mathscr{V}(R \bullet S) \doteq \{(x, y) \mid \exists z \,.\, (x, z) \in \mathscr{V}(R) \wedge (z, y) \in \mathscr{V}(S)\}$$

$$x \times y \subseteq R \bullet S \leftrightarrow \exists z \,.\, x \times z \subseteq R \wedge z \times y \subseteq S$$

Composition

$$x \subseteq A \ \wedge \ R \subseteq A \times B \ \rightarrow \ x \bullet R \subseteq B$$

$$y \subseteq B \ \wedge \ R \subseteq A \times B \ \rightarrow \ R \bullet y \subseteq A$$

$$\mathscr{V}(x \bullet R) \doteq \{(y) \mid (x, y) \in \mathscr{V}(R)\}$$

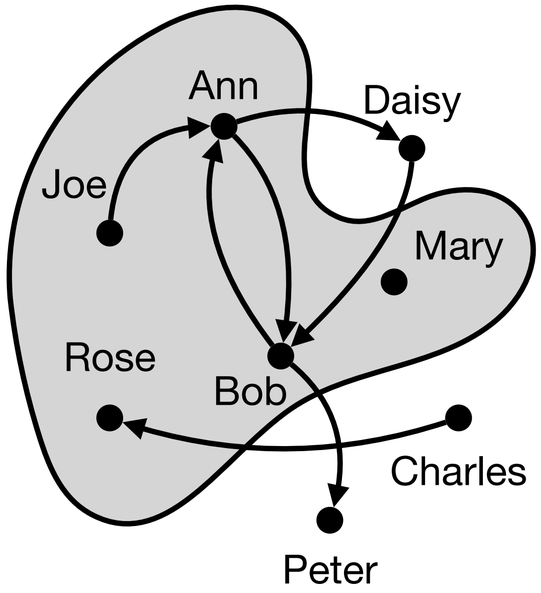
$$\mathscr{V}(R \bullet y) \doteq \{(x) \mid (x, y) \in \mathscr{V}(R)\}$$

$$y \subseteq x \bullet R \leftrightarrow x \times y \subseteq R$$

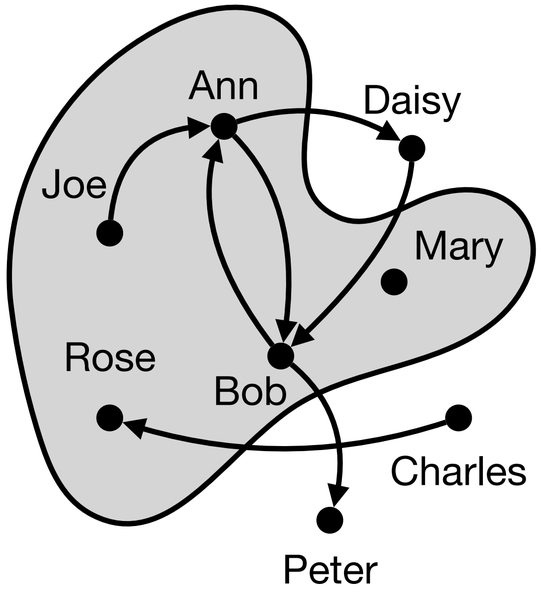
Composition

$$\text{Student} = \{(A), (M), (B), (R), (J)\}$$

$$\text{friend} = \{(J, A), (A, D), (A, B), (B, A), (B, P), (D, B), (C, R)\}$$



Composition

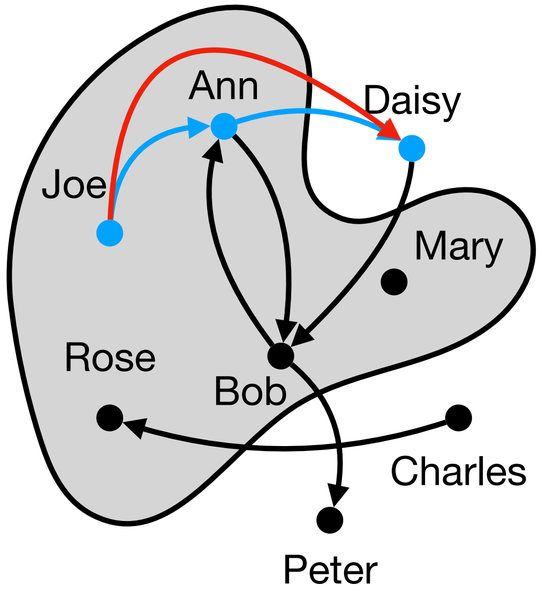


$$\text{Student} = \{(A), (M), (B), (R), (J)\}$$

$$\text{friend} = \{(J, A), (A, D), (A, B), (B, A), (B, P), (D, B), (C, R)\}$$

$$\text{friend} \bullet \text{friend} = \{(J, D), (J, B), (A, B), (A, P), (A, A), (B, B), (B, D), (D, A), (D, P)\}$$

Composition

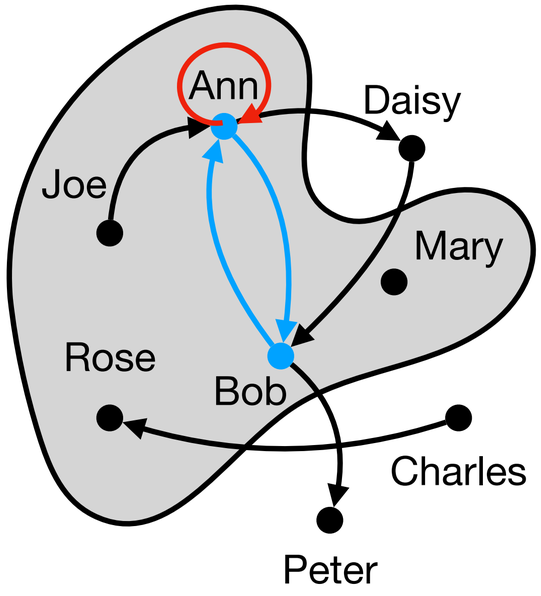


$$\text{Student} = \{(A), (M), (B), (R), (J)\}$$

$$\text{friend} = \{(J, A), (A, D), (A, B), (B, A), (B, P), (D, B), (C, R)\}$$

$$\text{friend} \bullet \text{friend} = \{(J, D), (J, B), (A, B), (A, P), (A, A), (B, B), (B, D), (D, A), (D, P)\}$$

Composition

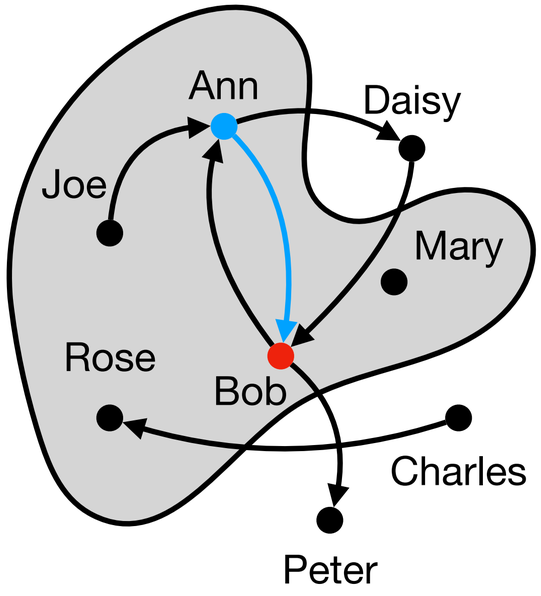


$$\text{Student} = \{(A), (M), (B), (R), (J)\}$$

$$\text{friend} = \{(J, A), (A, D), (A, B), (B, A), (B, P), (D, B), (C, R)\}$$

$$\text{friend} \bullet \text{friend} = \{(J, D), (J, B), (A, B), (A, P), (A, A), (B, B), (B, D), (D, A), (D, P)\}$$

Composition



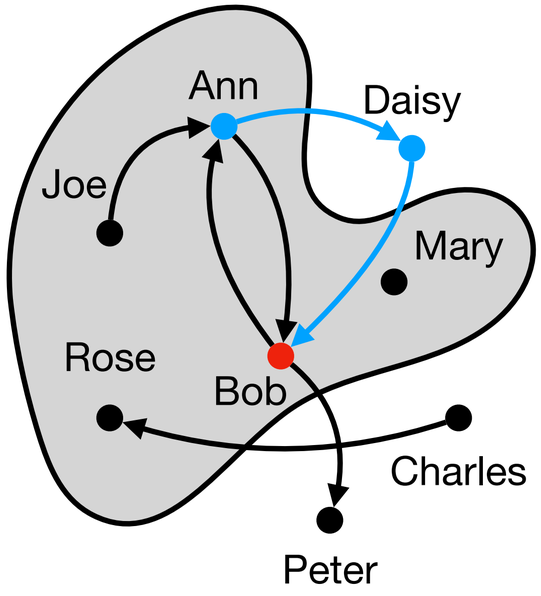
$$\text{Student} = \{(A), (M), (B), (R), (J)\}$$

$$\text{friend} = \{(J, A), (A, D), (A, B), (B, A), (B, P), (D, B), (C, R)\}$$

$$\text{friend} \bullet \text{friend} = \{(J, D), (J, B), (A, B), (A, P), (A, A), (B, B), (B, D), (D, A), (D, P)\}$$

$$\text{Ann} \bullet \text{friend} = \{(B), (D)\}$$

Composition



$$\text{Student} = \{(A), (M), (B), (R), (J)\}$$

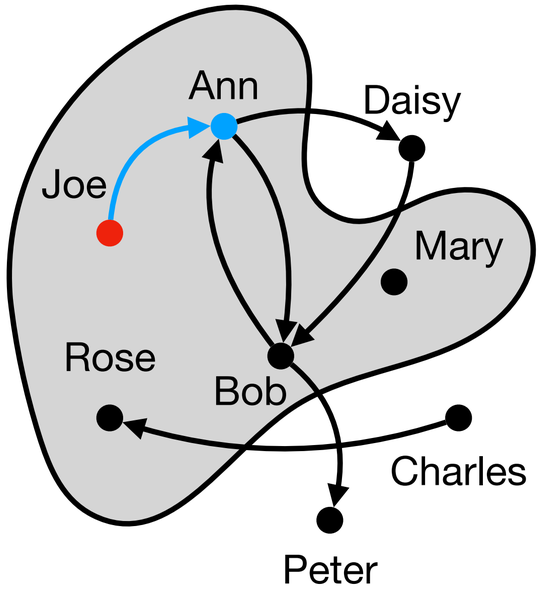
$$\text{friend} = \{(J, A), (A, D), (A, B), (B, A), (B, P), (D, B), (C, R)\}$$

$$\text{friend} \bullet \text{friend} = \{(J, D), (J, B), (A, B), (A, P), (A, A), (B, B), (B, D), (D, A), (D, P)\}$$

$$\text{Ann} \bullet \text{friend} = \{(B), (D)\}$$

$$\text{Ann} \bullet \text{friend} \bullet \text{friend} = \{(A), (B), (P)\}$$

Composition



$$\text{Student} = \{(A), (M), (B), (R), (J)\}$$

$$\text{friend} = \{(J, A), (A, D), (A, B), (B, A), (B, P), (D, B), (C, R)\}$$

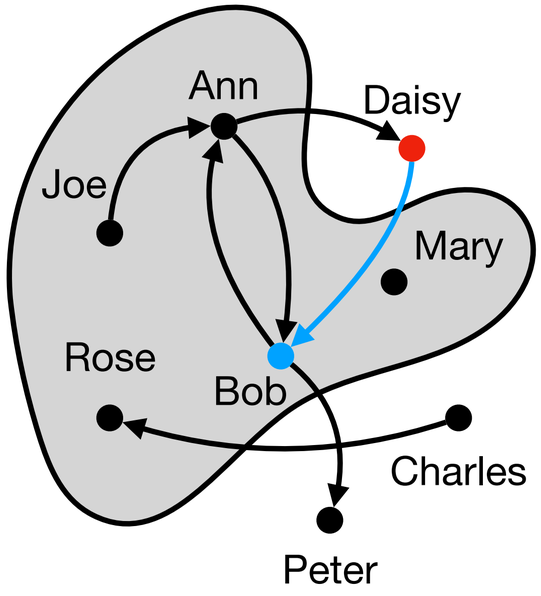
$$\text{friend} \bullet \text{friend} = \{(J, D), (J, B), (A, B), (A, P), (A, A), (B, B), (B, D), (D, A), (D, P)\}$$

$$\text{Ann} \bullet \text{friend} = \{(B), (D)\}$$

$$\text{Ann} \bullet \text{friend} \bullet \text{friend} = \{(A), (B), (P)\}$$

$$\text{friend} \bullet \text{Ann} = \{(J), (B)\}$$

Composition



$$\text{Student} = \{(A), (M), (B), (R), (J)\}$$

$$\text{friend} = \{(J, A), (A, D), (A, B), (B, A), (B, P), (D, B), (C, R)\}$$

$$\text{friend} \bullet \text{friend} = \{(J, D), (J, B), (A, B), (A, P), (A, A), (B, B), (B, D), (D, A), (D, P)\}$$

$$\text{Ann} \bullet \text{friend} = \{(B), (D)\}$$

$$\text{Ann} \bullet \text{friend} \bullet \text{friend} = \{(A), (B), (P)\}$$

$$\text{friend} \bullet \text{Ann} = \{(J), (B)\}$$

$$\text{friend} \bullet \text{Student} = \{(J), (A), (D), (B), (C)\}$$

FOL vs RL

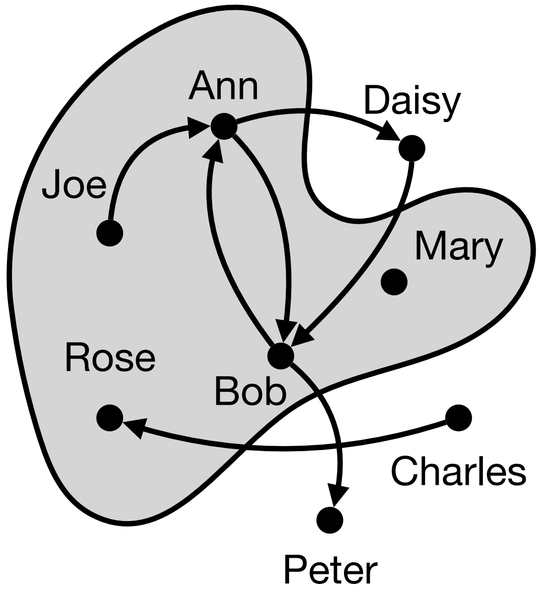
$$\forall x \,.\, \exists y \,.\, \text{Student}(y) \wedge \text{friend}(x, y)$$

$$\mathbf{U} \subseteq \text{friend} \bullet \text{Student}$$

$$\forall x \,.\, \forall y \,.\, \forall z \,.\, \text{friend}(x, y) \wedge \text{friend}(y, z) \rightarrow \text{friend}(x, z)$$

$$\text{friend} \bullet \text{friend} \subseteq \text{friend}$$

Transitive closure

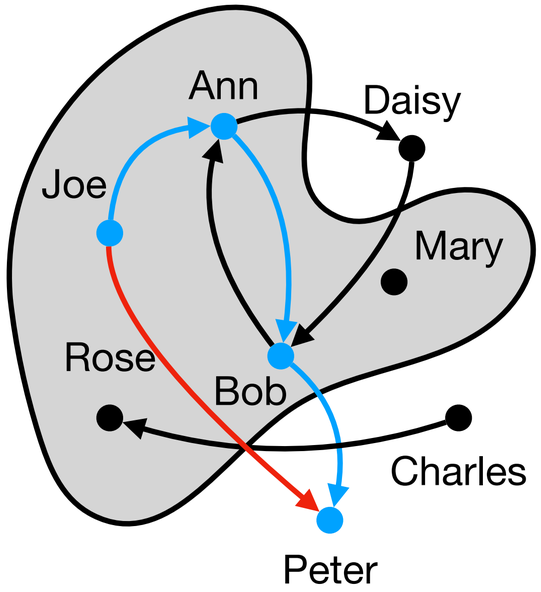


$$\text{Student} = \{(A), (M), (B), (R), (J)\}$$

$$\text{friend} = \{(J, A), (A, D), (A, B), (B, A), (B, P), (D, B), (C, R)\}$$

$$\text{friend}^{+} = \{(J, A), (J, D), (J, B), (J, P), (A, D), (A, B), (A, P), (A, A),$$
$$(B, A), (B, P), (B, D), (B, B), (D, B), (D, A), (D, D), (D, P), (C, R)\}$$

Transitive closure

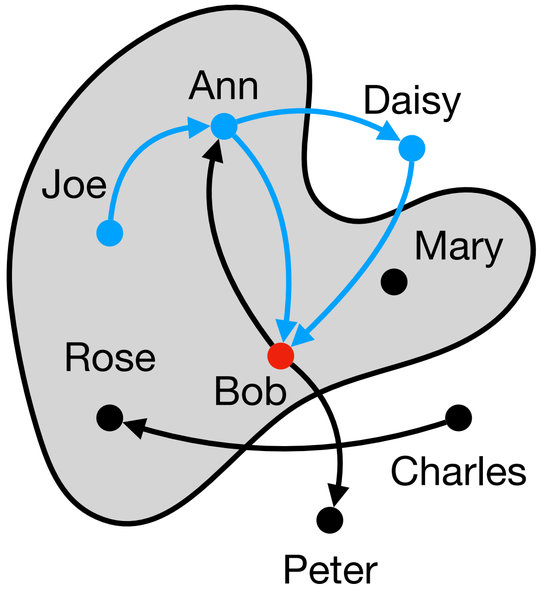


$$\text{Student} = \{(A), (M), (B), (R), (J)\}$$

$$\text{friend} = \{(J, A), (A, D), (A, B), (B, A), (B, P), (D, B), (C, R)\}$$

$$\text{friend}^{+} = \{(J, A), (J, D), (J, B), (J, P), (A, D), (A, B), (A, P), (A, A),$$
$$(B, A), (B, P), (B, D), (B, B), (D, B), (D, A), (D, D), (D, P), (C, R)\}$$

Transitive closure



$$\text{Student} = \{(A), (M), (B), (R), (J)\}$$

$$\text{friend} = \{(J, A), (A, D), (A, B), (B, A), (B, P), (D, B), (C, R)\}$$

$$\text{friend}^{+} = \{(J, A), (J, D), (J, B), (J, P), (A, D), (A, B), (A, P), (A, A),$$
$$(B, A), (B, P), (B, D), (B, B), (D, B), (D, A), (D, D), (D, P), (C, R)\}$$

$$\text{Joe} \bullet \text{friend}^{+} = \{(A), (D), (B), (P)\}$$

RL $\not\subseteq$ FOL

Ann is directly or indirectly a friend of everyone

$$\mathbf{U} \subseteq \text{Ann} \bullet \text{friend}^{+}$$

$$
\begin{aligned}
&\forall x \,.\, \text{friend}(A, x) \vee \\
&\quad (\exists y \,.\, \text{friend}(A, y) \wedge \text{friend}(y, x)) \vee \\
&\quad (\exists y, w \,.\, \text{friend}(A, y) \wedge \text{friend}(y, w) \wedge \text{friend}(w, x)) \vee \\
&\quad (\exists y, w, z \,.\, \text{friend}(A, y) \wedge \text{friend}(y, w) \wedge \text{friend}(w, z) \wedge \text{friend}(z, x)) \vee \\
&\quad \cdots
\end{aligned}
$$

RL in Alloy

$\varnothing$	**none**
U	**univ**
id	**iden**
$t \subseteq u$	t **in** u
$t \cup u$	t **+** u
$t \cap u$	t **&** u
$t \backslash u$	t **-** u
$t \times u$	t **->** u
$t \bullet u$	t **.** u
t°	**~**t
t^{+}	**^**t

Syntactic sugar

t = u

t != u

t **not in** u

no A

some A

lone A

one A

A <: R

R :> A

*R

all disj x,y : A | ϕ

some disj x,y : A | ϕ

t **in** u **and** u **in** t

not (t = u)

not (t **in** u)

A = **none**

A != **none**

all x,y : A | x = y

some A **and lone** A

R & (A -> **univ**)

R & (**univ** -> A)

^R + **iden**

all x,y : A | x != y **implies** ϕ

some x,y : A | x != y **and** ϕ

FOL vs RL

```
fact {
  // Each entry is contained in one directory
  all x : Entry | some y : Dir | y->x in contains
  all x : Entry, y,z : Dir {
    y->x in contains and z->x in contains implies y = z
  }
}

fact {
  // Each entry is contained in one directory
  all x : Entry | one contains.x
}
```

FOL vs RL

```
fact {
  // All directories are referred to in at most one entry
  all x : Dir, y,z : Entry {
    y->x in refersTo and z->x in refersTo implies y = z
  }
}

fact {
  // All directories are referred to in at most one entry
  all x : Dir | lone refersTo.x
}
```

FOL vs RL

```
fact {
  // The root is not referred in any entry
  all x : Entry, y : Root | x->y not in refersTo
}

fact {
  // The root is not referred in any entry
  no refersTo.Root
}
```

FOL vs RL

```
fact {
  // All non-root objects are referred to in at least one entry
  all x : Object | x not in Root implies some y : Entry | y->x in refersTo
}

fact {
  // All non-root objects are referred to in at least one entry
  Object-Root in Entry.refersTo
}
```

FOL vs RL

```
fact {
  // Different entries in a directory must have different names
  all x : Dir, y,z : Entry, w : Name {
    x->y in contains and x->z in contains and y->w in has and z->w in has implies y = z
  }
}

fact {
  // Different entries in a directory must have different names
  all d : Dir, n : Name | lone (d.contains & has.n)
}
```



Everyone has different styles

```
sig Person { style : one Style }
sig Style {}

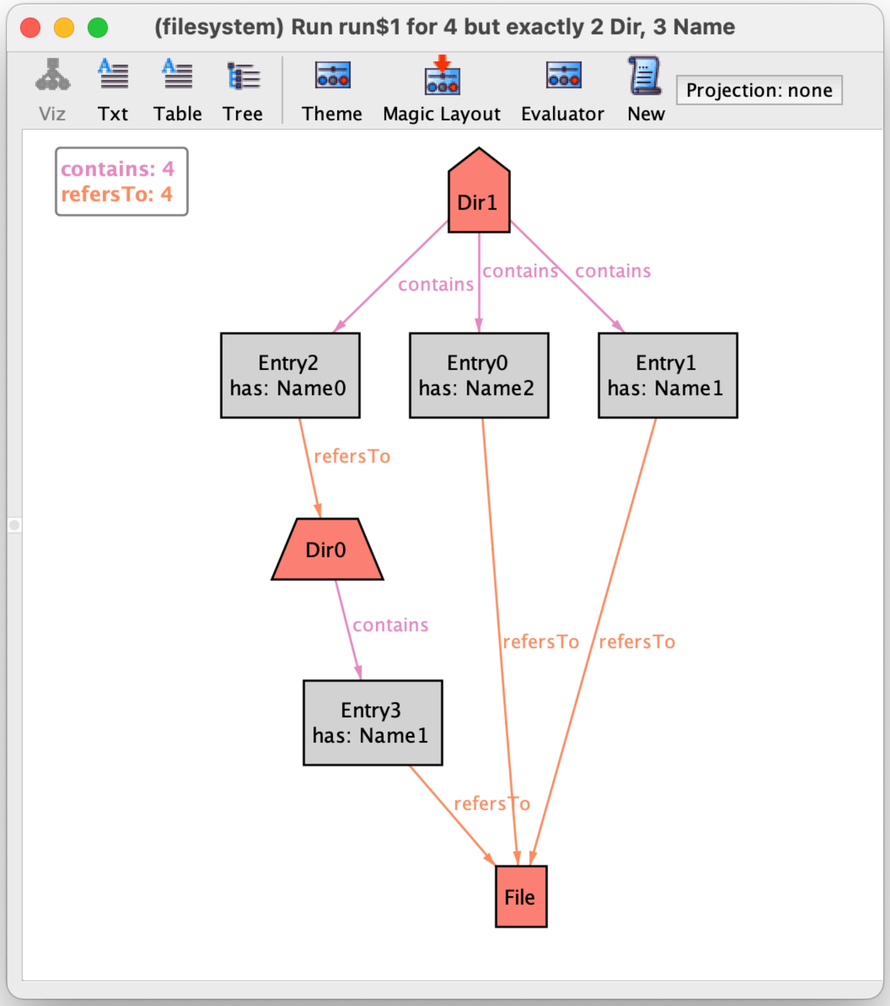
// First-order (point-wise) style
all x,y : Person, z : Style | x->z in style and y->z in style implies x=y

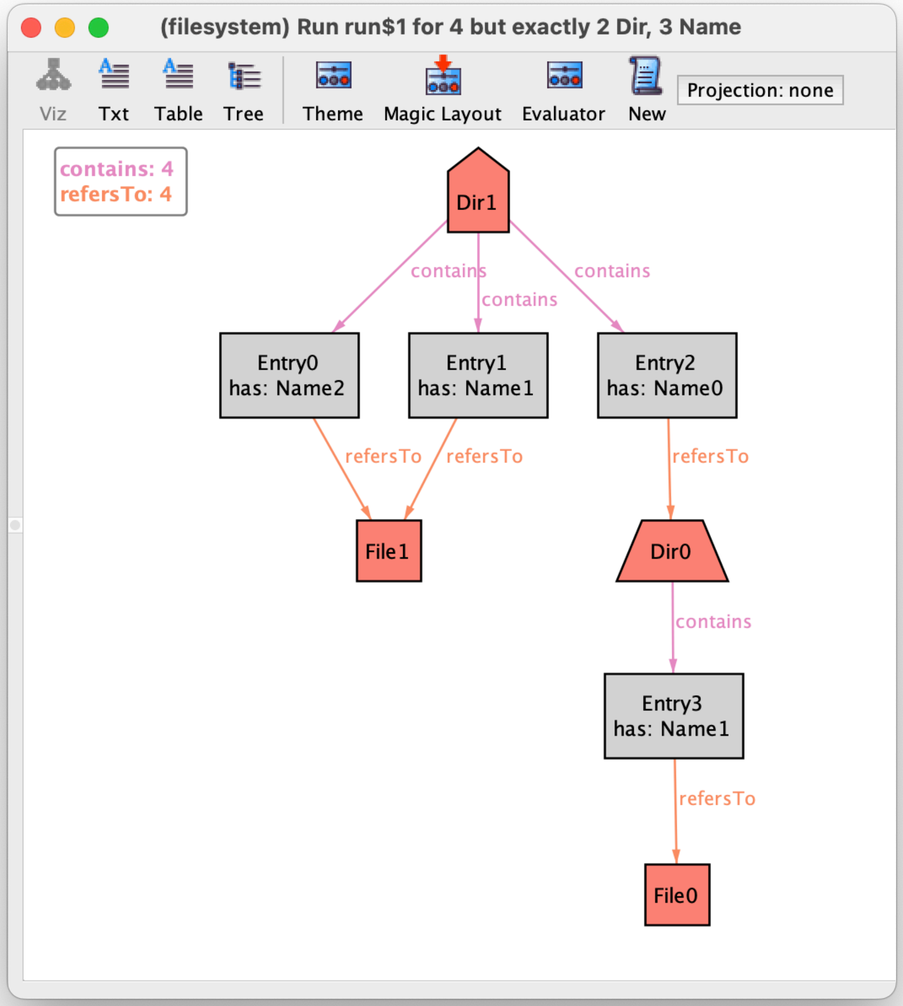
// Relational or navigational style
all z : Style | lone style.z

// Point-free style
style.~style in iden
```

Verification

Some instances





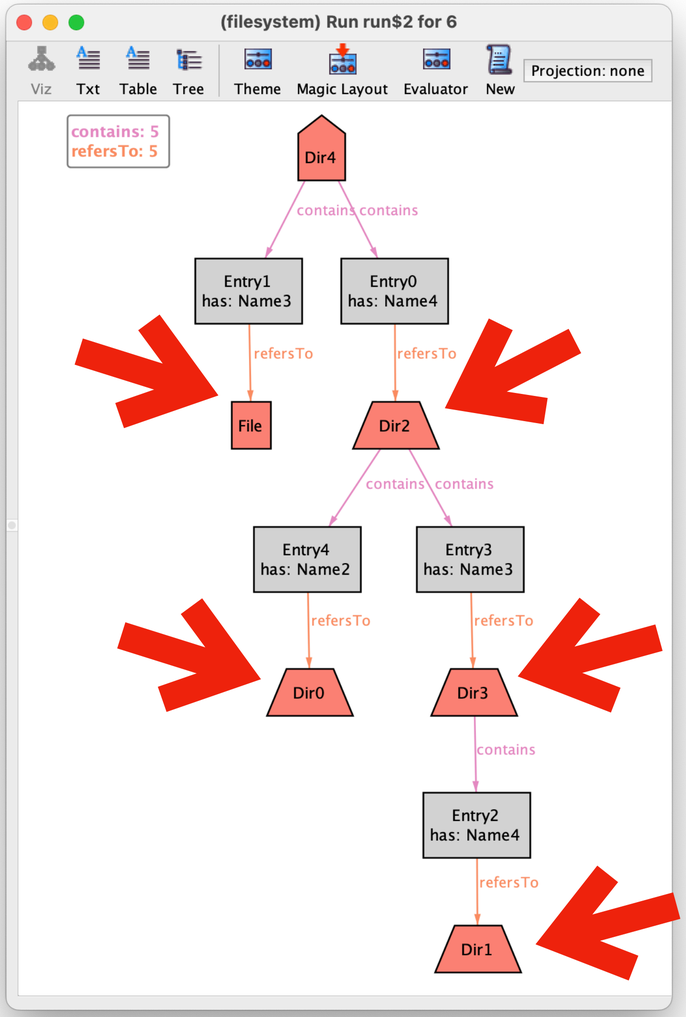
A desirable assertion

```
assert NoPartitions {
   // All objects are reachable from the root
   ???
}

check NoPartitions
```

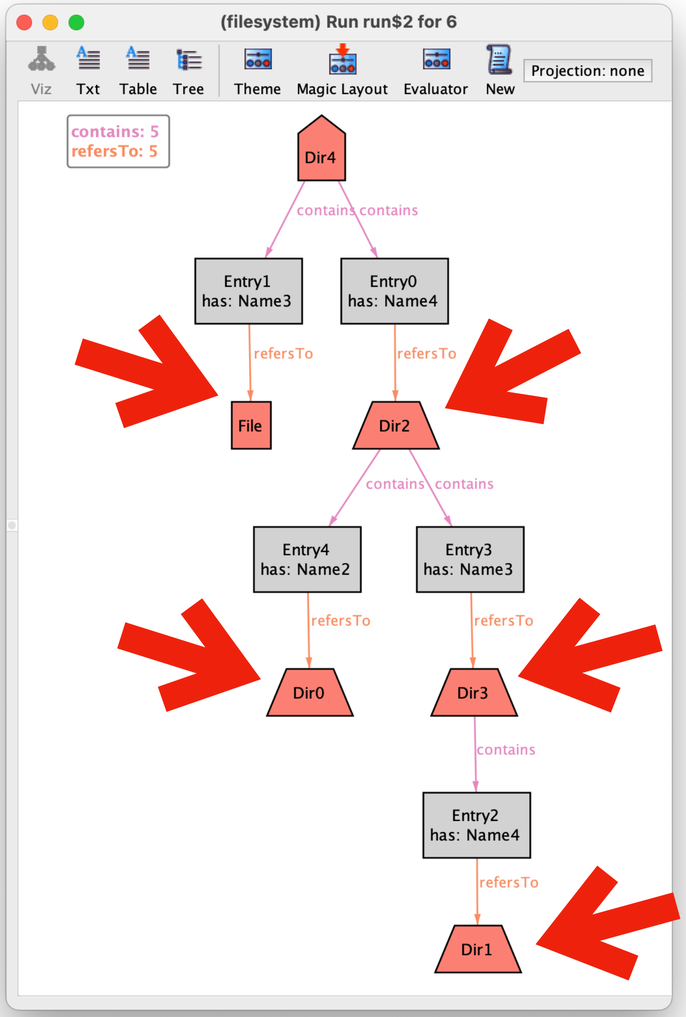
Reachable objects

`Root.contains` `Root.refersTo` `Root.contains.refersTo`



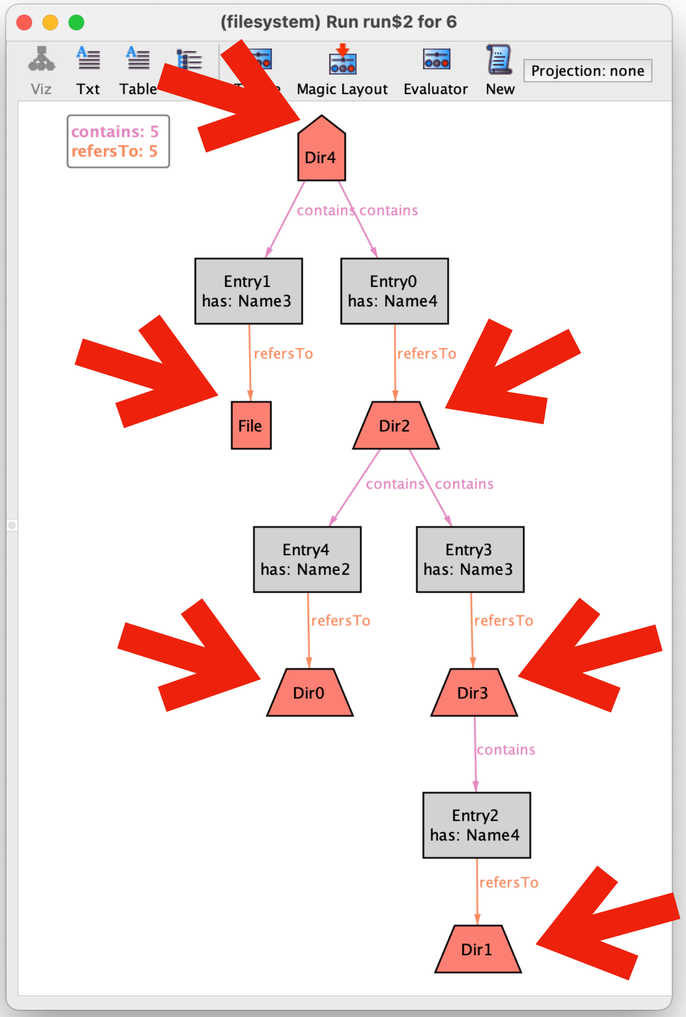
Reachable objects

`Root.^(contains.refersTo)`



Reachable objects

`Root.*(contains.refersTo)`

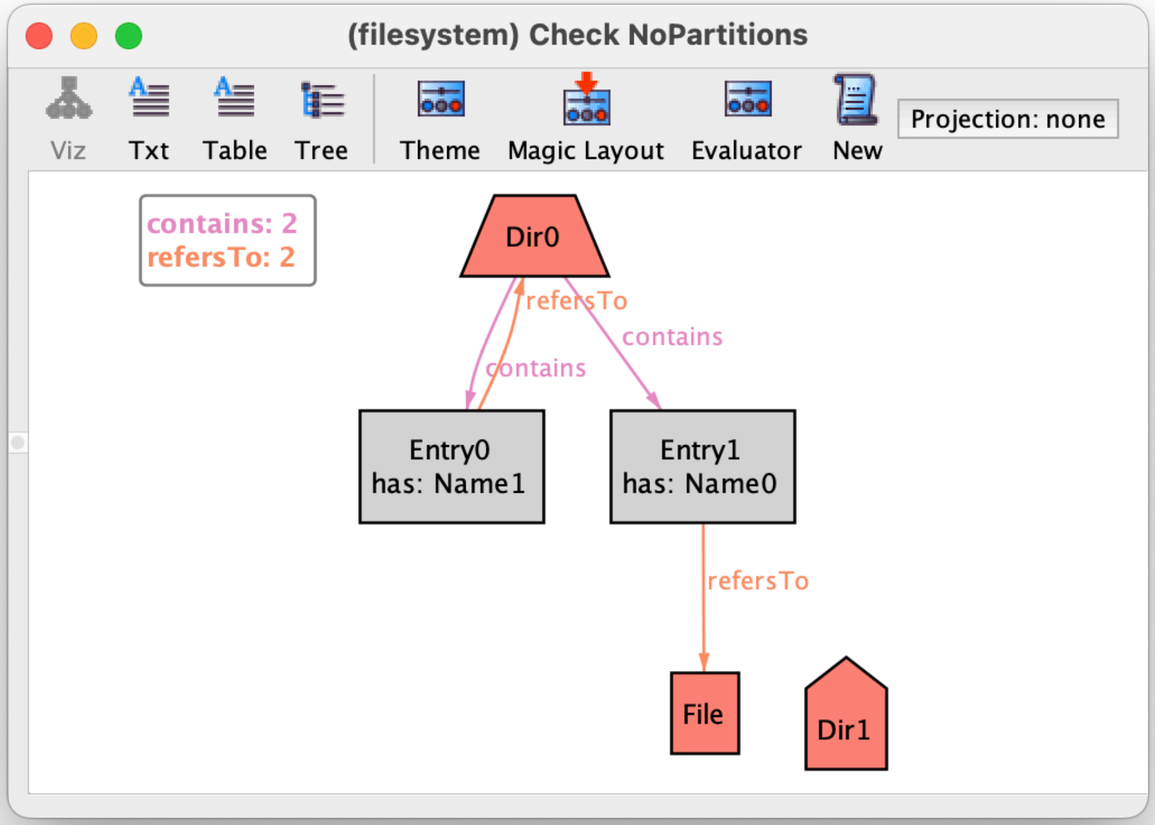


A desirable assertion

```
assert NoPartitions {
   // All objects are reachable from the root
   Object in Root.*(contains.refersTo)
}

check NoPartitions
```

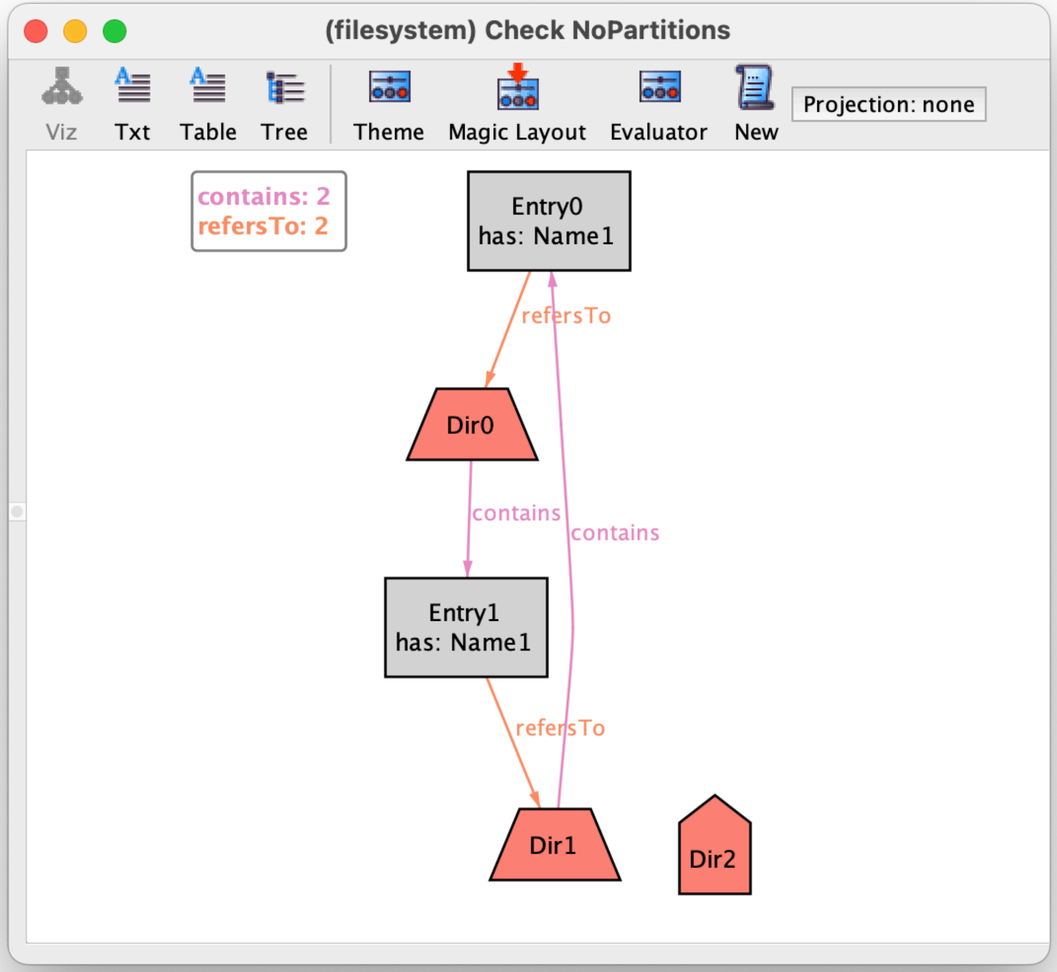
A counter-example



Missing requirement

```
fact {
  // A directory cannot be contained in itself
  all d : Dir | d not in d.contains.refersTo
}
```

Another counter-example



Missing requirement

```
fact {
  // A directory cannot be contained in itself
  all d : Dir | d not in d.^(contains.refersTo)
}
```

Missing requirement

```
fact {
  // A directory cannot be contained in itself
  no iden & ^(contains.refersTo)
}
```

```
Executing "Check NoPartitions"
   Solver=sat4j Bitwidth=4 MaxSeq=4 SkolemDepth=1 Symmetry=20 Mode=batch
   586 vars. 37 primary vars. 860 clauses. 3ms.
   No counterexample found. Assertion may be valid. 2ms.
```



Increasing confidence

- Increase the scope of **`check`** commands

```
check NoPartitions for 6
```

- Validation, validation, validation!
  - Specify more scenarios with **`run`** commands to validate the model
  - Check alternative equivalent specifications
  - Specify (apparently) trivial assertions (both valid and invalid)
  - …

Checking equivalent specifications

```
check {
  (all d : Dir | d not in d.^(contains.refersTo))
  iff
  (no iden & ^(contains.refersTo))
} for 4
```

